

AEGIS IDEA 3

รายงานอัปเดต v2

แก้ไขช่องโหว่จาก v1 (Timestamp Window + Secure Heartbeat) และสร้าง
Python SOC GUI Dashboard ครอบคลุม: HMAC Auth, Telegram สองภาษา,
Dead Man's Switch Countdown, UFW Containment, Incident Recovery Wizard

วันที่: 23 กรกฎาคม 2569 (2026-07-23)

ไฟล์หลัก: src/main.cpp (เฟิร์มแวร์ ESP32) + server_admin.py (Python SOC GUI)

Git commit ล่าสุด: c8f91d3

1. ภาพรวม: มีอะไรใหม่บ้างจาก v1

รายงานฉบับ v1 (22 ก.ค. 2569) จบด้วยการพบข้อสังเกต 2 จุดจากการทดสอบ MQTT จริงครั้งแรก คือ (1) เฟิร์มแวร์ยังไม่เช็ค timestamp window ทำให้คำสั่งเก่าผ่านได้ และ (2) heartbeat ยังไม่มี HMAC ป้องกัน รายงานฉบับนี้ (v2) สรุปสิ่งที่ทำต่อจากจุดนั้น:

- แก้เฟิร์มแวร์ทั้ง 2 จุดที่พบ แล้วทดสอบซ้ำ 5 สถานการณ์บนฮาร์ดแวร์จริงอีกครั้ง — ผ่านครบ 5/5
- สร้าง Python SOC GUI Dashboard (server_admin.py) ขึ้นมาใหม่ทั้งหมด เป็นศูนย์ควบคุมฝั่งแอดมิน แทนการยิงคำสั่งด้วยสคริปต์ทดสอบ
- ต่อยอดฟีเจอร์ตามเอกสารออกแบบที่ยังไม่เคย implement จริง: UFW Auto-Block คู่กับการตัดวงจร, Incident Recovery Wizard ตามขั้นตอน 5.4, Telegram แจ้งเตือนสองภาษาแบบมืออาชีพ, และตัวนับถอยหลัง Dead Man's Switch แบบเรียลไทม์
- แก้บั๊กเสียงแจ้งเตือนที่ใช้งานไม่ได้ (pygame.mixer ขาดไลบรารีระบบ) และปรับ UI ทั้งหมดให้เป็น dashboard มืออาชีพ

สรุปสถานะ ณ วันนี้: ระบบความปลอดภัยหลัก (HMAC/Replay/Tamper/Stale/Dead Man's Switch) ผ่านการทดสอบจริงครบ 100% ส่วน GUI ควบคุมสร้างเสร็จและทดสอบ syntax/เปิดใช้งานได้ แต่บางฟีเจอร์ (UFW live click, Auto-Detector) ยังรอการยืนยัน/ยังไม่ได้ทำ — ดูหัวข้อ 6

2. แก้ไขช่องโหว่ที่พบใน v1 (เฟิร์มแวร์ ESP32 รอบ 2)

2.1 เพิ่มการตรวจสอบ Timestamp Window

เพิ่มค่าคงที่ MAX_COMMAND_AGE_SEC = 30 และเรียก NTP sync ตอนบูต (syncTimeNTP()) เพื่อให้ time(nullptr) ของบอร์ดตรงกับเวลาจริง แล้วเช็คก่อนอนุมัติคำสั่งทุกครั้ง:

```
if (abs((long)(now - ts)) > MAX_COMMAND_AGE_SEC) {
    sendAck("STALE", "timestamp out of window");
    return;
}
```

2.2 เพิ่ม HMAC ให้ Heartbeat

เดิม heartbeat รับข้อความอะไรก็ได้โดยไม่ตรวจสอบ เปลี่ยนให้ต้องมีรูปแบบเดียวกับคำสั่งคือ HMAC + nonce + timestamp:

```
{"hb": "alive", "nonce": "<>", "ts": <>, "sig": "<HMAC-SHA256>"}
```

payload ที่เซ็นคือ hb + "|" + nonce + "|" + ts เหมือนกับคำสั่ง CUT_UPLINK/RESTORE_UPLINK ทุกประการ ทำให้ผู้ที่เข้าถึง MQTT broker แต่ไม่รู้ secret key ไม่สามารถปลอมสัญญาณ heartbeat เพื่อขัดขวาง Dead Man's Switch ได้อีกต่อไป

3. ผลทดสอบซ้ำ 5 สถานการณ์ (ครั้งที่ 2 — บนฮาร์ดแวร์จริง)

ยังคำสั่งชุดเดิมผ่าน mosquitto_pub/sub อีกครั้งหลังแก้ไขเฟิร์มแวร์ ผลลัพธ์ทั้งหมดตรงตามทีออกแบบไว้แล้ว:

Test 1: Valid CUT_UPLINK ผ่าน

```
ACK: {"ack": "OK", "detail": "uplink cut"}
```

Test 2: Replay Attack ผ่าน

```
ACK: {"ack": "REPLAY", "detail": "nonce reused"}
```

Test 3: Tamper Attack ผ่าน

```
ACK: {"ack": "BAD_HMAC", "detail": "sig mismatch"}
```

Test 4: Stale Timestamp (>30s) ผ่าน

```
ACK: {"ack": "STALE", "detail": "timestamp out of window"}
```

Test 5: Dead Man's Switch ผ่าน

```
status: {"state": "LOCKDOWN", "reason": "DEAD MAN'S SWITCH..."}
```

หมายเหตุระหว่างทดสอบ

ระหว่างชุดทดสอบพบเหตุการณ์ MQTT reconnect เกิดขึ้นเอง 1 ครั้ง (บอร์ดส่ง status "ONLINE" แทนที่จะเป็น ACK ที่คาดหวัง) ตรวจสอบด้วยการส่งคำสั่งซ้ำและเฝ้าดู 20 วินาทีโดยไม่แตะ serial เลย พบว่าบอร์ดกลับมาเสถียรและตอบสนองถูกต้องปกติ คาดว่าเป็นการสะดุดของ WiFi ชั่วคราว ไม่ใช่บั๊กของ firmware logic

4. Python SOC GUI Dashboard (server_admin.py)

สร้างโปรแกรม Desktop GUI (Tkinter) ขึ้นมาใหม่ทั้งหมด ทำหน้าที่เป็น "เครื่องมือฝังแอดมิน" แทนที่การยิงคำสั่งด้วยสคริปต์ทดสอบแบบ manual เดิม โครงสร้างประกอบด้วย 3 ส่วนหลัก:

- MQTTManager — เชื่อมต่อ broker, subscribe ACK/STATUS, ส่งสัญญาณ heartbeat แบบเซ็น HMAC ทุก 15 วินาทีในแรมพื้นหลัง
- SQLite Audit Trail (aegis_audit.db) — บันทึกทุกเหตุการณ์ (คำสั่งที่ส่ง, ACK ที่ได้รับ, การเปลี่ยนสถานะ, การพยายามใส่ PIN ผิด, ขั้นตอน recovery) พร้อมปุ่ม Export เป็น CSV
- Admin PIN Authentication — ปุ่มควบคุมที่มีผลจริง (ตัด/ต่อ uplink) ต้องใส่ PIN ก่อนทุกครั้ง ไล่ผิดจะถูกบันทึกเป็น SECURITY_ALERT

รายละเอียดฟีเจอร์แต่ละตัวที่เพิ่มเข้ามาระหว่างเซสชันนี้ อยู่ในหัวข้อ 5

5. รายละเอียดฟีเจอร์ที่พัฒนาในเซสชันนี้

5.1 ปรับ UI เป็น Professional Dashboard เสร็จแล้ว

จากเดิมหน้าต่างเดียว 750x700 เรียงปุ่ม/ป้ายซ้อนกันแน่น เปลี่ยนเป็นเลย์เอาต์ 2 คอลัมน์ (header + sidebar ควบคุม + log panel ฟังก์ชัน + footer) ขนาด 1180x760 ปรับขนาดได้ (resizable) มี Section/Card component แยกกลุ่มฟีเจอร์ชัดเจน พร้อมนาฬิกาเรียลไทม์และแถบสรุปกลไกความปลอดภัยที่ footer

5.2 แก้ปัญหาเสียงแจ้งเตือนไม่ทำงาน

เสร็จแล้ว

พบ 3 บั๊กซ้อนกัน: pygame.mixer ใช้ไม่ได้ (ระบบไม่มี libSDL2_mixer), ลืม import os, และไม่มีไฟล์เสียงอยู่จริง แก้โดยเลิกพึ่ง pygame.mixer เปลี่ยนไปเล่นผ่าน paplay/aplay ที่มีอยู่แล้วในเครื่อง (รันใน background thread ไม่บล็อก GUI) และสร้างไฟล์ Sound.wav (ไซเรนสองโทนเสียง) ขึ้นเองด้วย Python ล้วนๆ ทดสอบเล่นจริงผ่านแล้ว

5.3 Telegram Alert แบบสองภาษา มีอาชีวะ

เสร็จแล้ว

จากเดิมส่งข้อความบรรทัดเดียวภาษาไทยล้วน ไม่มีเวลา/ข้อมูลเพิ่มเติม เปลี่ยนเป็น template ที่มีหัวข้อระบบ, ไอคอนสถานะสี (สีแดง = LOCKDOWN / สีเขียว = RESTORED), เหตุผลในกล่อง code block, เวลา, RSSI/Heap, และคำแนะนำสองภาษา (EN แล้วตาม TH) เพิ่มการแจ้งเตือนตอน RESTORED ด้วย (เดิมมีแค่ตอน LOCKDOWN) เพื่อปิดลูปการแจ้งเตือน — ทดสอบส่งจริงผ่าน Telegram Bot API แล้ว ได้ผลลัพธ์ ok:true ทั้งสองกรณี

5.4 Live Dead Man's Switch Countdown

เสร็จแล้ว

เพิ่มการ์ดแสดงวินาทีที่เหลือก่อน ESP32 จะตัด uplink เองถ้าขาด heartbeat จำนวนจากเวลาที่ heartbeat ล่าสุด ถูกส่งสำเร็จจริง (เช็คสถานะการเชื่อมต่อ ไม่ใช่แค่พยายามส่ง) นับถอยหลังทุกวินาที เปลี่ยนสี เขียว/เหลือง/แดง ตามความเสี่ยง (60→30 5 = เขียว, 30→10 5 = เหลือง, ต่ำกว่า 10 5 = แดง) ทดสอบแล้วนับถอยหลังทำงานถูกต้องจริง

5.5 UFW Auto-Block คู่กับการตัดวงจร

ยังไม่ทดสอบครบ

ตามเอกสารข้อ 5.2 ขั้น 3 ที่ต้องบล็อกซอฟต์แวร์ (UFW) คู่กับตัดกายภาพ เพิ่ม prompt ถาม IP ผู้บุกรุกทันทีหลังกด CUT_UPLINK (เว้นว่างได้ถ้าไม่ทราบ) แล้วสั่ง ufw deny ผ่าน pkexec (popup ขอรหัสผ่านกราฟิกของ KDE polkit ที่ตรวจสอบแล้วว่า ทำงานอยู่บนเครื่องนี้) รันใน background thread ไม่บล็อก GUI ระหว่างรอผู้ใช้ยืนยันตัวตน

5.6 Incident Recovery Wizard

ยังไม่ทดสอบครบ

หน้าต่างแยกที่พาไล่ทำตามลำดับกู้คืนระบบ 5 ขั้นตามเอกสารข้อ 5.4 ครบถ้วน: (1) ยืนยัน Out-of-band Access ผ่าน Management VLAN (2) บล็อก IP ถาวรผ่าน UFW (3) ส่ง RESTORE_UPLINK ผ่าน MQTT ต้องใส่ PIN (4) Reload UFW เปิดบริการ กลับคืน (5) กรอกสรุปบทเรียนแล้วปิดเหตุการณ์ — แต่ละขั้นมีสถานะ Pending/Done และบันทึก log audit ทุกขั้น ทดสอบเปิด หน้าต่างและแสดงผลถูกต้องแล้ว

หมายเหตุสถานะ "ยังไม่ทดสอบครบ" (5.5, 5.6): ผมทดสอบได้แค่ compile ผ่าน เปิดหน้าต่างได้ และ logic ถูกต้องตามที่ออกแบบ แต่ขั้นตอนที่ต้องกรอกรหัสผ่าน root จริงผ่าน popup (pkexec) เป็นสิ่งที่ผมกดยืนยันแทนไม่ได้ ต้องให้ผู้ใช้งานทดลองเองเพื่อยืนยัน ว่า flow ทำงานจบครบวงจรจริง

6. ข้อยกเว้นและสิ่งที่ยังไม่ได้ทำ (Known Gaps)

6.1 ยังไม่มี Auto-Detector — ระบบยังไม่รู้ IP ผู้บุกรุกเอง

คำถามสำคัญที่เกิดขึ้นระหว่างพัฒนา: "ระบบจะรู้ได้ยังไงว่า IP ไหนคือแฮกเกอร์?" คำตอบตรงไปตรงมาคือ ตอนนี้อย่างไรก็ตาม ฟังก์ชัน UFW Auto-Block (5.5) ต้องให้แอดมินพิมพ์ IP เองเสมอ เพราะตัวตรวจจับภัยจริงตามเอกสาร (Suricata บน NAS) ยังไม่มี NAS จริง (ตรวจสอบแล้ว: sshd บนเครื่องนี้ปิดอยู่ ไม่มี log การโจมตีให้เผ่าดูด้วย) มี 2 ทางเลือกที่คุยกันไว้แต่ยังไม่ได้ตัดสินใจทำ:

- ทางเลือก A: เปิด sshd จริงแล้วเฝ้า journalctl ดัก brute-force จริง — สมจริงสุดแต่เปิดช่องโหว่จริงบนเครื่องไม่แนะนำสำหรับ demo
- B (): Test Stale/Tampered

6.2 ยังไม่ได้ต่อสาย LAN จริงเข้า Relay

ทุกการทดสอบจนถึงตอนนี้เป็นการยืนยัน logic ผ่าน log/LED/relay click เท่านั้น ยังไม่เคยตัดสายอินเทอร์เน็ตจริงเพื่อพิสูจน์ว่า uplink ขาดจริงเมื่อ relay ทำงาน

6.3 Git — Commit แล้ว แต่ Push ไม่สำเร็จ

commit ล่าสุด (c8f91d3) รวมงานทั้งหมดในเซสชันนี้ไว้ในเครื่องเรียบร้อยแล้ว แต่ push ขึ้น GitHub (repo private: Kittipat050871/NETWORK-SEC-Project) ไม่สำเร็จเพราะเครื่องนี้ไม่มี credential ตั้งค่าไว้ในระดับที่ automation เข้าถึงได้ ต้องให้ผู้ผู้ใช้ push เองผ่าน terminal หรือ VS Code

7. สรุปสถานะรวมและขั้นตอนถัดไป

สถานะปัจจุบัน (23 ก.ค. 2569)

- ความปลอดภัยระดับ protocol (HMAC/Nonce/Timestamp/Dead Man's Switch) ผ่านการทดสอบจริงครบ 5/5 บนฮาร์ดแวร์ ยืนยันสองรอบ
- Python SOC GUI Dashboard + +
- + audit log
- commit push GitHub

แนะนำขั้นตอนถัดไป

- Push โค้ดขึ้น GitHub ให้เสร็จ (ผู้ใช้ดำเนินการเอง)
- UFW Block/Reload Incident Recovery Wizard end-to-end
- Auto-Detector (A B 6.1)
- LAN / uplink (Latency)
- (AEGIS_System_Design.docx) IDEA 3 "expected" "confirmed"